

GRADUAÇÃO

 FIAP

MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 05

Organizando o acesso a dados remotos

Do código funcional à arquitetura de dados escalável

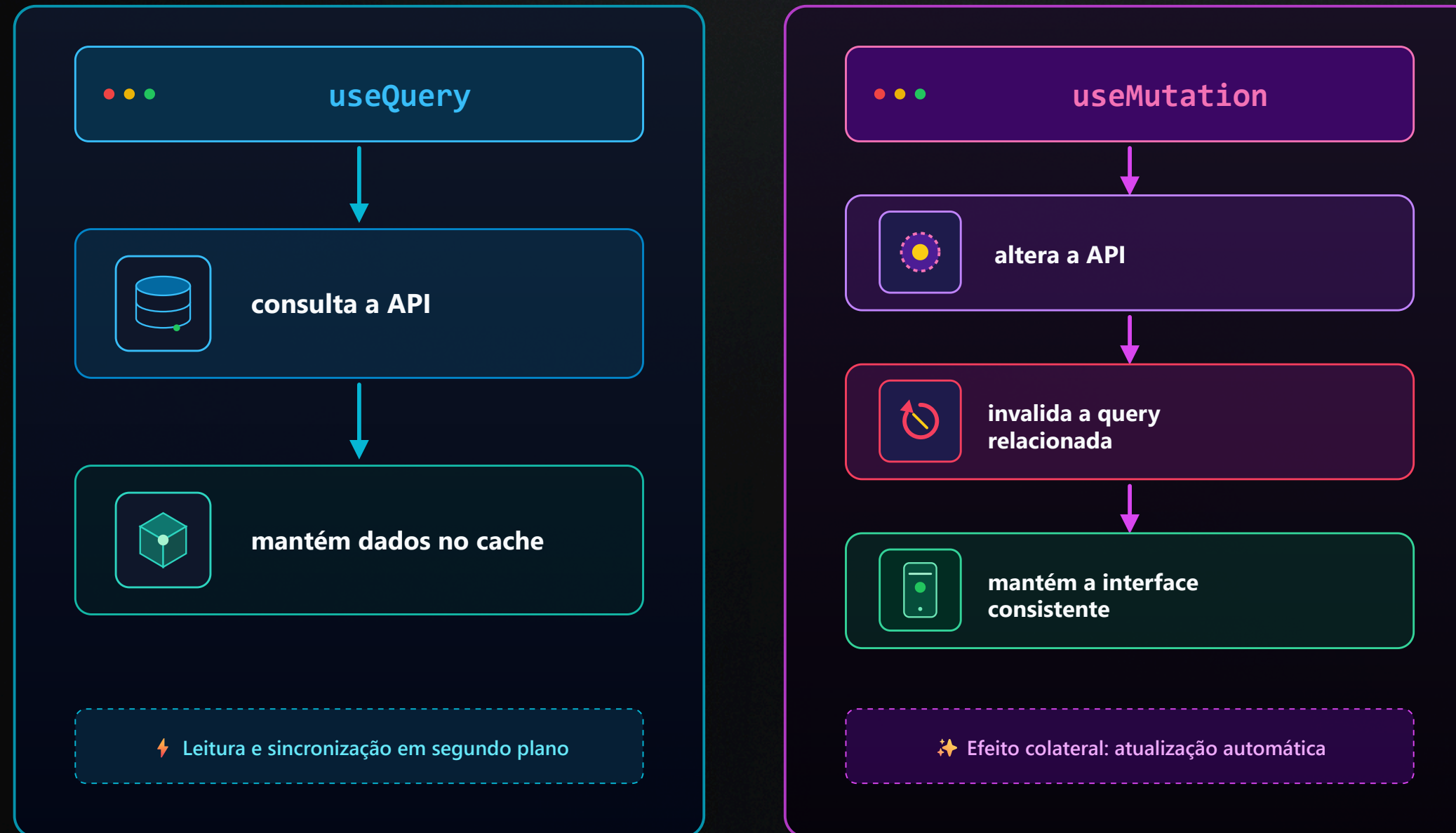


“Dividir cada uma das dificuldades que examinasse em tantas parcelas quantas fosse possível e necessário para melhor resolvê-las.”

— René Descartes, Discurso do Método



O CICLO REMOTO JÁ ESTÁ COMPLETO



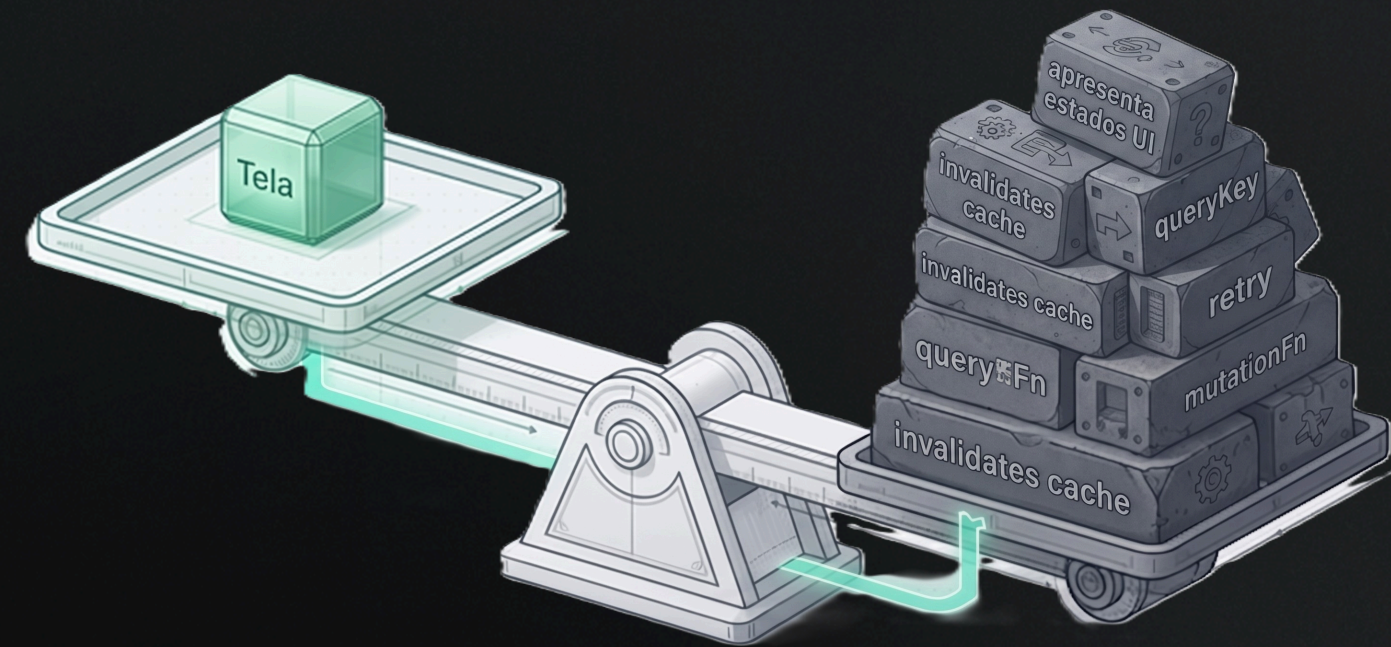
O DESAFIO AGORA DEIXA DE SER FAZER FUNCIONAR.

O CÓDIGO CRESCER JUNTO COM O COMPORTAMENTO

```
export default function TweetsScreen() {  
  const queryClient = useQueryClient();  
  
  const tweetsQuery = useQuery({  
    queryKey: ["tweets"],  
    queryFn: getTweets,  
  });  
  
  const createTweetMutation = useMutation({  
    mutationFn: createTweet,  
    retry: false,  
    onSuccess: () => {  
      queryClient.invalidateQueries({ queryKey: ["tweets"] });  
    },  
  });  
  
  // estados, eventos e interface  
}
```

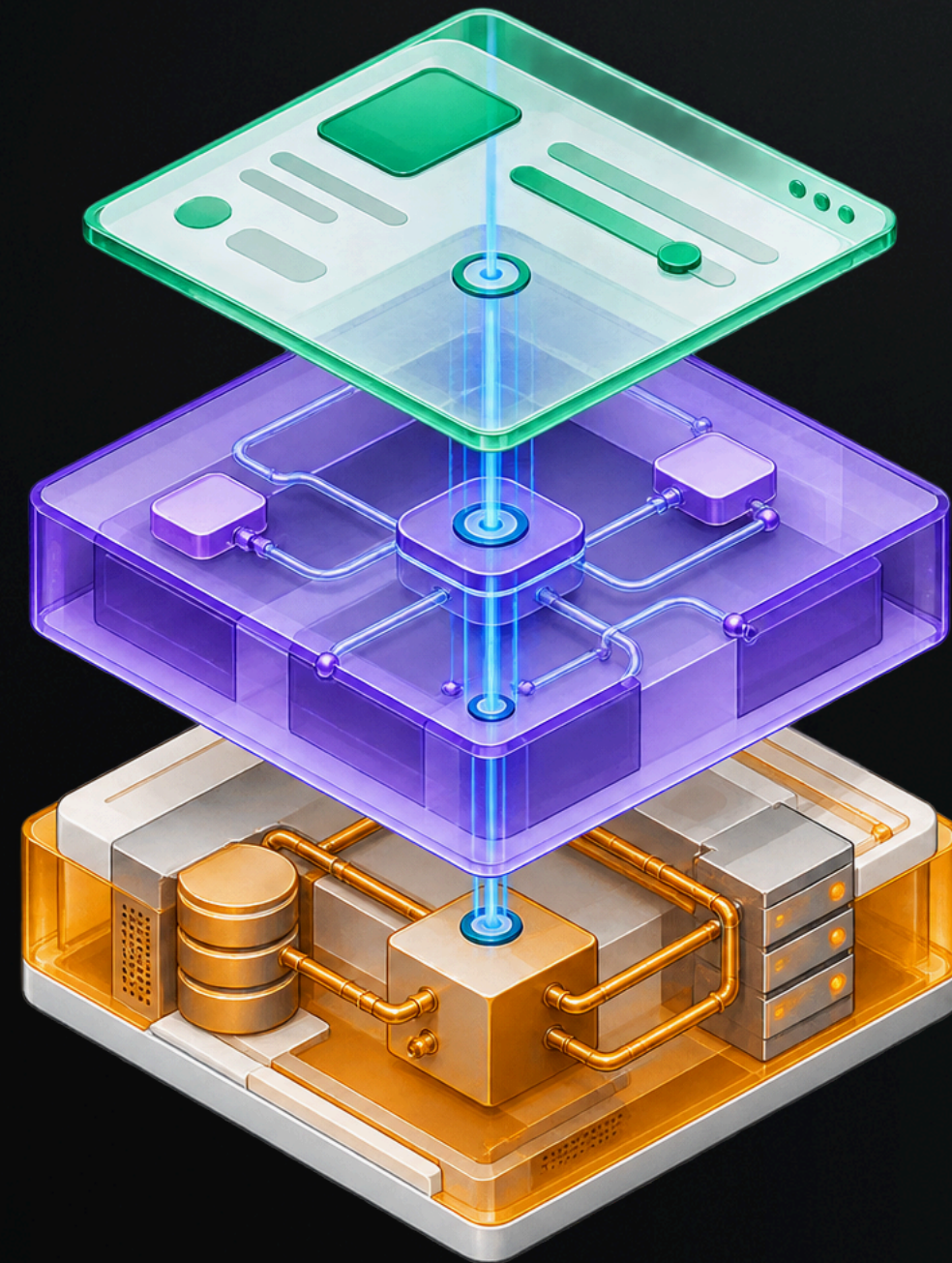
A TELA CONHECE DETALHES DE MAIS

- conhece a `queryKey` ;
- escolhe a `queryFn` ;
- configura a `mutationFn` ;
- define o `retry`;
- invalida o cache;
- trata eventos do usuário;
- apresenta os estados na interface.



O CÓDIGO FUNCIONA, MAS RESPONSABILIDADES DIFERENTES OCUPAM O MESMO LUGAR.

A SEPARAÇÃO COMEÇA PELAS RESPONSABILIDADES



A Tela

Apresenta estados, gerencia a interface visual e reage às ações do usuário.

Exemplo: O que o usuário vê.

O Custom Hook

Coordena o gerenciamento do ciclo de dados remotos e integra com o TanStack Query.

Exemplo: `useMoviesQuery()`

O Serviço

Realiza a comunicação HTTP crua e o transporte de rede.

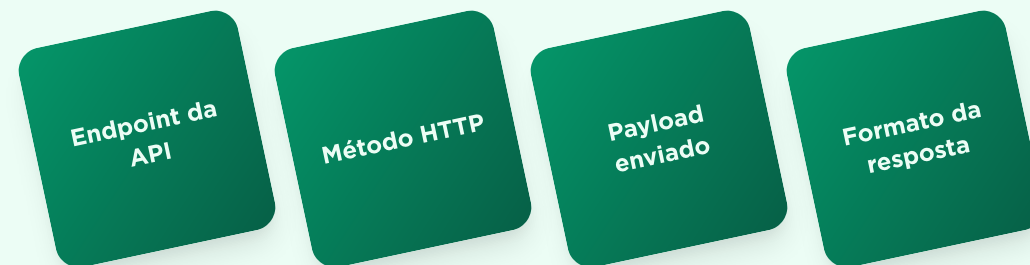
Exemplo: `getMovies()`

SERVIÇO VS. CUSTOM HOOK

O Serviço

Função Principal: Transportar os dados.

O que conhece:



O Custom Hook

Função Principal: Gerencia o ciclo remoto.

O que conhece:



UM CUSTOM HOOK COMPÕE HOOKS EXISTENTES

```
export function useTweetsQuery() {  
  return useQuery({  
    queryKey: ["tweets"],  
    queryFn: getTweets,  
  });  
}
```

`useTweetsQuery()` não cria um novo mecanismo de estado.

Ele utiliza o mecanismo existente e fornece uma operação própria da aplicação.

ABSTRAIR É ELEVAR A LINGUAGEM DA APLICAÇÃO

DETALHES TÉCNICOS

```
useQuery({  
  queryKey: ["tweets"],  
  queryFn: getTweets,  
});
```

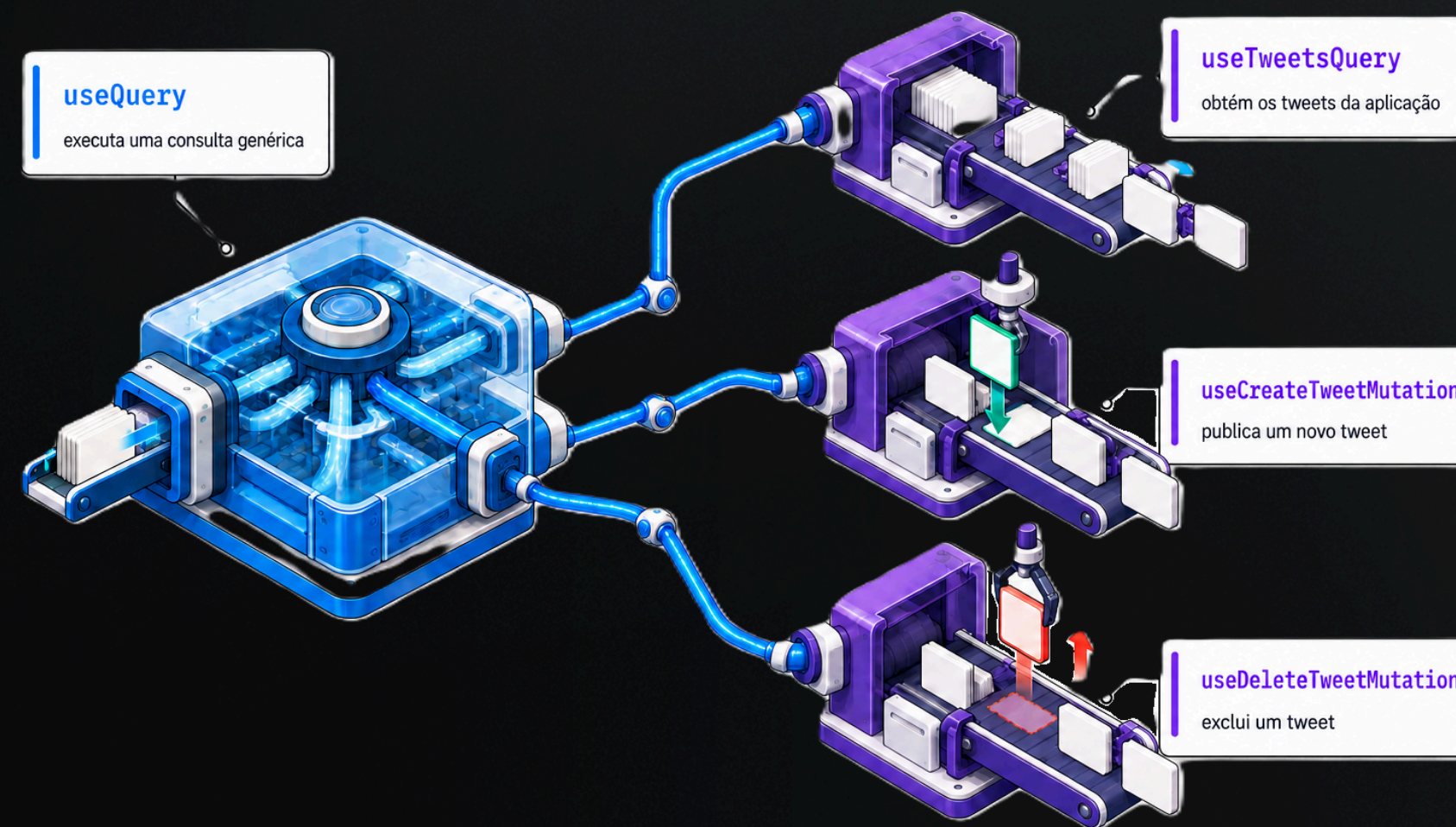
INTENÇÃO DA APLICAÇÃO

```
useTweetsQuery();
```

O ganho principal não está na quantidade de linhas.

O GANHO ESTÁ NA LINGUAGEM UTILIZADA PELA TELA.

O NOME PASSA A FAZER PARTE DO MODELO DA APLICAÇÃO



O nome comunica a intenção antes de revelar a implementação.

O CUSTOM HOOK ESTABELECE UM CONTRATO

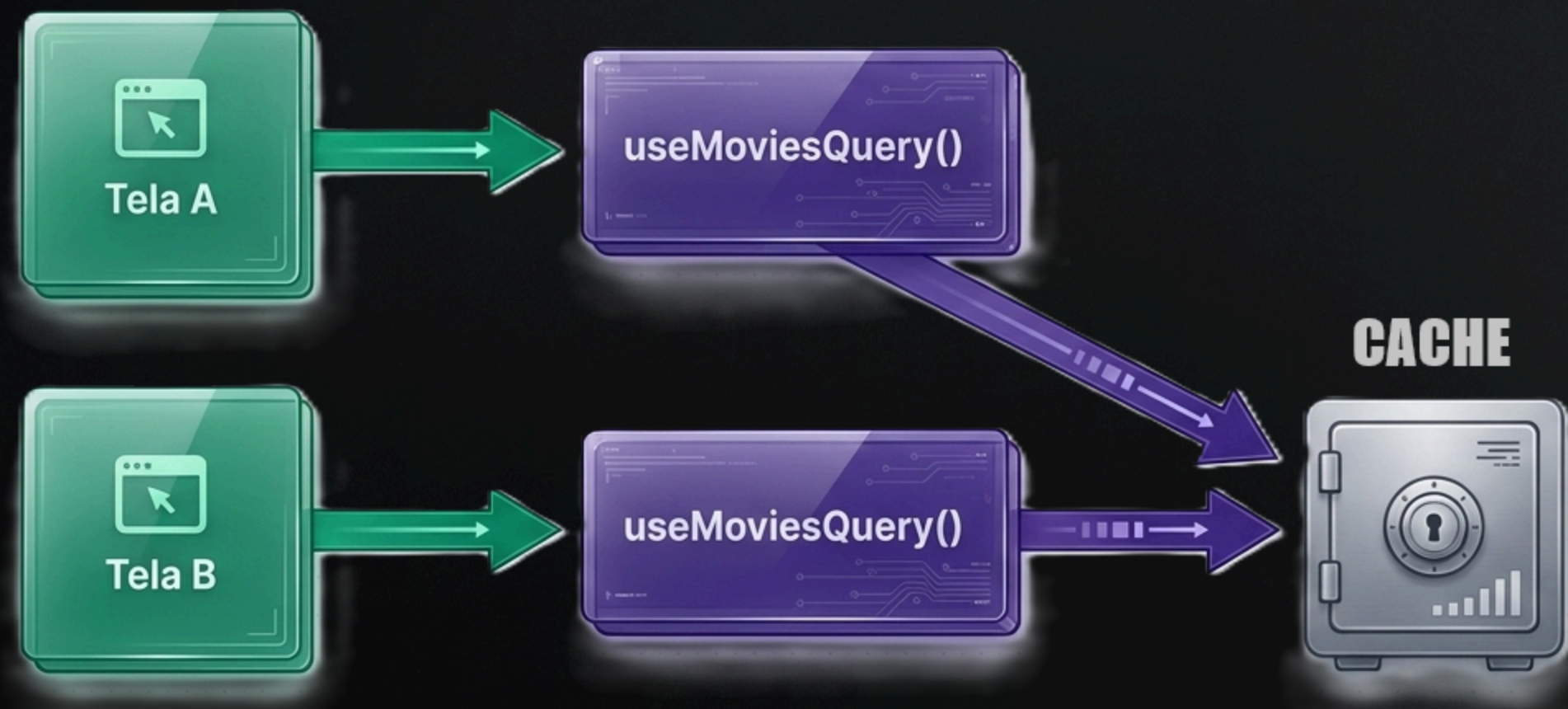
```
const tweetsQuery = useTweetsQuery();
```

A tela recebe:

```
data  
isPending  
isError  
isSuccess  
refetch
```

A CONFIGURAÇÃO INTERNA PODE EVOLUIR SEM ALTERAR A FORMA DE USO NA TELA.

CUSTOM HOOKS REUTILIZAM LÓGICA, NÃO CRIAM UM ESTADO GLOBAL



O CUSTOM HOOK COMPARTILHA A FORMA DE ACESSAR O DADO.

O TanStack Query compartilha o dado por meio da `queryKey` e do cache.

A CONFIGURAÇÃO GANHA UM PONTO DE REFERÊNCIA

Sem custom hook, cada tela pode definir:

- uma queryKey diferente
- um staleTime diferente
- um retry diferente
- uma invalidação diferente

Com `useTweetsQuery()`, as telas utilizam a mesma decisão de acesso remoto.

- menos divergência → comportamento mais previsível

A ABSTRAÇÃO PERMANECE FINA

O CUSTOM HOOK MANTÉM

- `useQuery` OU `useMutation`;
- chaves e funções;
- `retry`;
- invalidação;
- efeitos relacionados ao dado remoto.

A TELA MANTÉM

- apresentação de loading e erro;
- alertas e mensagens;
- navegação;
- eventos do usuário;
- componentes visuais.

CADA AÇÃO MANTÉM SUA PRÓPRIA MUTATION

```
useCreateTweetMutation();  
useUpdateTweetMutation();  
useDeleteTweetMutation();
```



CADA AÇÃO POSSUI UMA FUNÇÃO, UM ESTADO E EFEITOS POSTERIORES INDEPENDENTES.

AS REGRAS DOS HOOKS CONTINUAM VÁLIDAS

- inicie o nome com `use` ;
- chame os custom hooks diretamente no corpo do componente;
- nunca chame dentro de condições, repetições ou eventos;
- utilize o hook em um componente ou em outro hook;
- mantenha a responsabilidade declarada pelo nome.

```
const deleteTweetMutation = useDeleteTweetMutation();
```

UM HOOK OCUPA UM ARQUIVO COM INTENÇÃO EXPLÍCITA

```
src/  
├── app/  
│   └── tweets.tsx  
├── hooks/  
│   ├── useTweetsQuery.ts  
│   ├── useCreateTweetMutation.ts  
│   ├── useUpdateTweetMutation.ts  
│   └── useDeleteTweetMutation.ts  
└── services/  
    └── tweets.ts
```

A estrutura acompanha as operações ensinadas.

A QUERY DEIXA A TELA

```
// hooks/useTweetsQuery.ts

import { useQuery } from "@tanstack/react-query";
import { getTweets } from "@services/tweets";

export function useTweetsQuery() {
  return useQuery({
    queryKey: ["tweets"],
    queryFn: getTweets,
  });
}
```

A MUTATION MANTÉM A INVALIDAÇÃO PRÓXIMA DA ALTERAÇÃO

```
// hooks/useCreateTweetMutation.ts

export function useCreateTweetMutation() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: createTweet,
    retry: false,
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["tweets"] });
    },
  });
}
```

A TELA PASSA A DECLARAR INTENÇÕES

```
export default function TweetsScreen() {  
  const tweetsQuery = useTweetsQuery();  
  const createTweetMutation = useCreateTweetMutation();  
  
  function handlePublish(tweet: NewTweet) {  
    createTweetMutation.mutate(tweet);  
  }  
  
  // estados, eventos e interface  
}
```

O COMPORTAMENTO PERMANECE. A LEITURA DO CÓDIGO MUDA.



DESAFIO



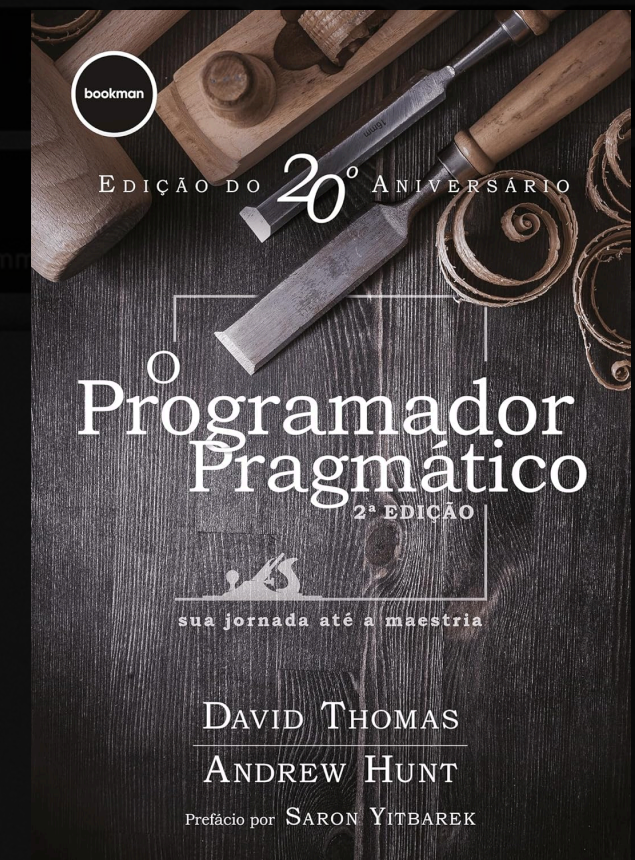
RECOMENDAÇÃO DE LEITURA

Um código bem organizado permite alterar uma decisão sem espalhar mudanças por toda a aplicação.

O Programador Pragmático – David Thomas e Andrew Hunt

✨ Por que ler?

-  Mostra como dividir o software em partes com responsabilidades bem definidas
-  Apresenta formas de reduzir dependências desnecessárias entre diferentes partes do código
-  Incentiva a criação de abstrações que expressem intenção, não apenas detalhes técnicos
-  Desenvolve o raciocínio necessário para construir sistemas que possam evoluir com segurança



LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026